SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

Element

Combine Anything: An On-Device AI Crafting Game for iOS

Mini Project Report

Submitted by

Pradyum Mistry

Roll No. 16010123236

B.Tech Computer Engineering, Last Year

K. J. Somaiya School of Engineering, Mumbai

Mini Project, Semester VII, 2026-2027

October 2026

Source code: github.com/altf4-games/Element

Abstract

Element is an iOS crafting game in the style of Little Alchemy and Infinite Craft. The player drags two elements together and gets a new one. Past a small set of pinned starting recipes, every result is generated by Apple's on-device Foundation Models framework, so the game has no server, no API key and no network traffic.

This report describes how the app is built. SwiftUI draws a drag and drop canvas, a searchable discovery log and a map screen. One `@Observable` view model owns the game state. A `@Generable` struct makes the model return a name, an emoji, a colour and a description, and the app cleans each field before use. Core Data stores every discovered element and every generated combination, so a pair that has been answered once is never asked again. If the model is unavailable, slow, or only repeats an input, a hand-written recipe book with 125 recipes answers instead, and a combination never ends in an error. The app has two worlds, Classic and Periodic, each with its own prompt, starting elements and saved progress.

I did not measure the quality of the model's answers, and the project has no automated tests. The report says what the design guarantees and what it does not.

Keywords: iOS, SwiftUI, Foundation Models, on-device AI, Core Data, guided generation, crafting game.

Table of contents

1. Introduction **5**
1.1 Background 5
1.2 Problem statement 5
1.3 Objectives 5
1.4 Scope and method 5
2. Tools and technologies **7**
2.1 Why on-device generation 7
2.2 Technology stack 7
3. System design **8**
3.1 Architecture 8
3.2 Source layout 8
3.3 Data model 9
3.4 Game modes 10
3.5 State management 10
4. The combination engine **11**
4.1 From drop to result 11
4.2 Prompt design 11
4.3 Guided generation 11
4.4 Cleaning the model's output 12
4.5 Cache and pinned recipes 12
4.6 Handling failure 12
4.7 Concurrency and sessions 13
5. User interface **14**
5.1 Screen map 14
5.2 Craft screen 14
5.3 Discoveries and detail 15
5.4 Atlas 15
5.5 Settings and accessibility 15
6. Results **16**
6.1 The finished app 16
6.2 Development timeline 17
6.3 What the design guarantees 17
7. Limitations and future work **19**
8. Conclusion **20**
Appendix A: Classic system prompt **21**
Appendix B: Combine function, abridged **22**
References **23**

List of figures

Figure 1: Architecture of Element 8
Figure 2: The Core Data model 9
Figure 3: How a combination is resolved 11
Figure 4: Screens and how they connect 14
Figure 5: Classic mode: Craft, Discoveries and Atlas 16
Figure 6: Mode switcher, Periodic Craft and Periodic Discoveries 17

List of tables

Table 1: Frameworks and tools used in Element 7
Table 2: Source files and their roles 8
Table 3: Differences between the two game modes 10
Table 4: Where each kind of state lives 10
Table 5: What the app does when generation goes wrong 13
Table 6: AI creativity setting 13
Table 7: Craft screen gestures 14
Table 8: Project history from the repository 17

1. Introduction

1.1 Background

Crafting games ask the player to combine two things and see what comes out. Little Alchemy [1] does this with a fixed list of recipes, so every player who mixes fire and water gets steam and every pair either has an answer or does not. Infinite Craft [2] removed that limit by letting a language model decide what a pair makes, so the game has no final element.

Until recently, a game like that needed a server. Each combination meant a request to a hosted model, which costs money per call and needs a network connection. Apple's Foundation Models framework in iOS 26 [3] gives apps direct access to the language model that runs on the device itself. An app can ask it questions without an API key, a backend or a connection.

1.2 Problem statement

I wanted to find out whether a crafting game in the style of Infinite Craft could run completely on the phone. That raises three questions:

1. How do I get short, usable answers (a name, an emoji, a colour and a description) from a language model, and not paragraphs of text?
2. What should the game do when the model is unavailable, slow, or returns something unusable?
3. How do I keep the player's discoveries across launches, so the game does not forget what it has already invented?

1.3 Objectives

1. Build an iOS app in SwiftUI where the player drags two elements together and gets a new one.
2. Generate new elements with the on-device Foundation Models framework, using guided generation so the output has a fixed structure.
3. Make sure a combination never ends in an error, by adding an offline fallback.
4. Store discoveries and combinations with Core Data, so repeat pairs are answered from the cache.
5. Offer two worlds, Classic and Periodic, each with its own starting elements, prompt and progress.
6. Add a searchable discovery log with favourites and JSON export, and a map screen that shows where each element was discovered.

1.4 Scope and method

Element started as a SwiftUI mini project in August 2026. A second round of work in October rebuilt the Craft screen in the style of Infinite Craft and added the offline fallback, and a third added the Periodic mode. The app has no backend and no third party dependencies. It uses only Apple frameworks and is about 2,700 lines of Swift in 16 files.

This report describes the design and the code as they stand in the repository [8]. The screenshots were taken in the iOS Simulator. I did not measure the quality of the model's answers, and the app has no automated tests. Section 7 says what that means for the results.

2. Tools and technologies

2.1 Why on-device generation

Running the model on the device has three practical effects for this project. There is nothing to host, so the app can be built and run from Xcode alone. There are no API keys or per-request costs. And a player's combinations never leave the phone, which is relevant because the app saves every pair they try.

The cost is that the app depends on Apple Intelligence. It needs a device or Simulator host that supports it, with the model downloaded. The app checks this at launch and says so on the Craft screen when the model is not ready (Section 4.6).

2.2 Technology stack

Table 1: Frameworks and tools used in Element

Technology	Used for
Swift 5 and SwiftUI [4]	All screens, layout, gestures, animation and navigation
Foundation Models (iOS 26) [3]	On-device language model, `LanguageModelSession`, `@Generable` guided generation
Observation [5]	`@Observable` view model shared through the SwiftUI environment
Core Data [6]	Saving discovered elements and cached combinations; `@FetchRequest` for the log
MapKit [7] and Core Location	The Atlas map, pins, and the optional Locate Me button
`@AppStorage`, `@SceneStorage`	Settings, selected tab, tray sort order and game mode
`os.Logger`	Logging model failures that the game recovers from silently
Xcode 26, iOS 26.5 target	Build, Simulator and previews

The project has no Swift packages. Dragging, haptics and animation use SwiftUI's own `DragGesture`, `sensoryFeedback` and `symbolEffect` modifiers.

3. System design

3.1 Architecture

The app has three layers. SwiftUI views draw the screens and send the player's actions to one view model. The view model, `GameViewModel`, owns the discovered elements and the combination cache, and decides where each answer comes from. Under it sit three sources: the Foundation Models session, the offline `RecipeBook`, and Core Data.

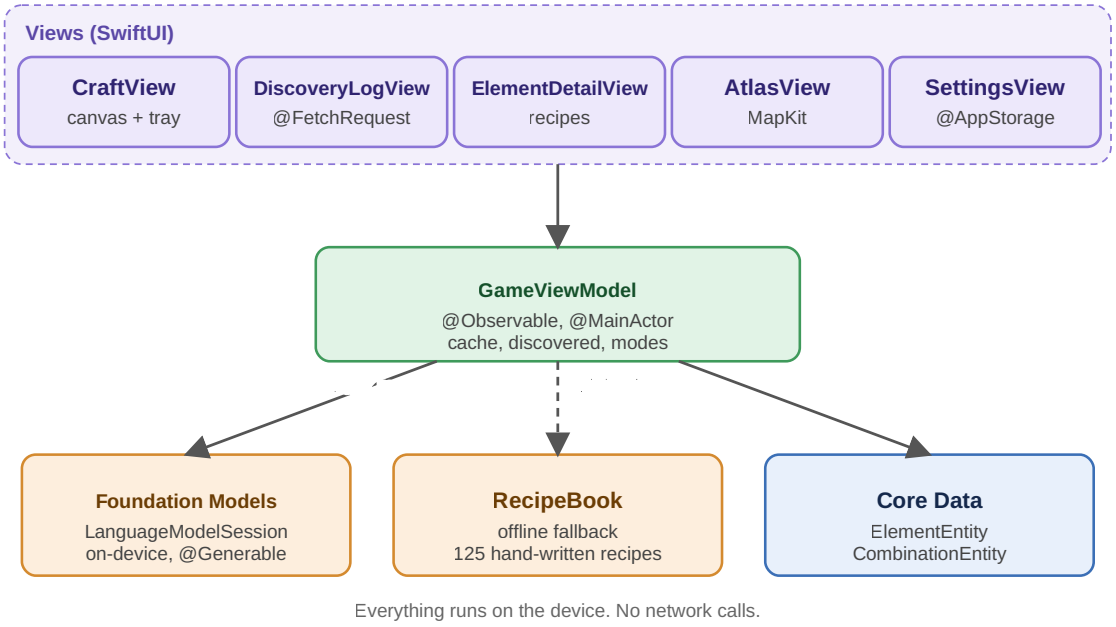


Figure 1: Architecture of Element

The view model is the single source of truth for game state. Views that only need the current list read it from the environment. The Discoveries tab is the exception: it reads Core Data directly through `@FetchRequest`, which keeps the list in sync with the store without extra code.

3.2 Source layout

Table 2: Source files and their roles

File	Lines	Role
CraftView.swift	773	Drag and drop canvas, tray, search and sort, merge animation, toast
GameViewModel.swift	377	State, combine pipeline, cache, Core Data reads and writes
RecipeBook.swift	303	Offline recipes and the rules for improvised results
GameMode.swift	167	Classic and Periodic: starting elements, pinned recipes, prompts

File	Lines	Role
SettingsView.swift	150	Theme, game mode, AI creativity, reduce motion, reset
Element.swift	141	`Element` model, `GeneratedElement`, input cleaning
DiscoveryLogView.swift	138	Searchable log, favourites filter, JSON export
ElementDetailView.swift	127	One element, the recipes that make it and what it makes
ElementPill.swift	116	Emoji and name chip, `FlowLayout` for the expanded tray
ElementTile.swift, Blob.swift	171	Shared tiles, animated background shapes
AtlasView.swift, LocationManager.swift	138	Map screen and location permission
ContentView, ElementApp, Persistence	105	Tab container, app entry, Core Data stack

3.3 Data model

Core Data holds two entities. `ElementEntity` is one discovered element. `CombinationEntity` is one cached pair and what it produced. The cache key is the two names, lowercased and sorted, joined with a plus sign, so Fire + Water and Water + Fire share one entry. Both entities carry a `mode` attribute, which is how Classic and Periodic keep separate progress in one store.

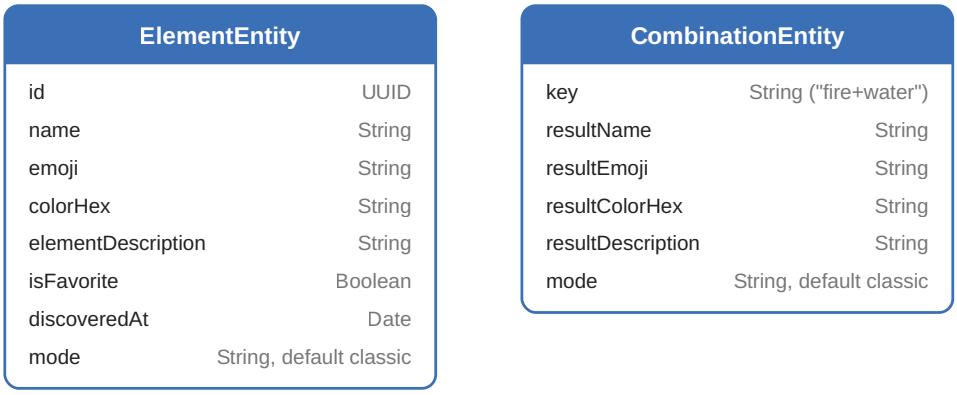


Figure 2: The Core Data model

In memory the app uses a plain `Element` struct that is `Codable`, `Hashable` and compared by lowercased name. That is what the UI sees, so no view touches a managed object directly except the Discoveries list.

3.4 Game modes

A `GameMode` value bundles everything that differs between the two worlds. Switching mode saves the choice to `UserDefaults`, clears the in-memory cache, and reloads the elements and combinations for the new mode from Core Data. Progress in the other mode is not touched.

Table 3: Differences between the two game modes

	Classic	**Periodic**
Starting elements	Fire, Water, Earth, Air	Hydrogen, Oxygen, Carbon, Nitrogen, Sodium, Chlorine, Iron, Sulfur
Pinned recipes	6 (Fire + Water = Steam, Water + Earth = Plant, ...)	8 (Hydrogen + Oxygen = Water, Sodium + Chlorine = Salt, Iron + Carbon = Steel, ...)
Offline recipes	93	32
Prompt goal	The most obvious everyday result	A real compound, mineral, alloy or material
Description style	One playful sentence	Chemical formula first, then one short sentence
Discovery toast	NEW DISCOVERY	NEW COMPOUND

3.5 State management

Table 4: Where each kind of state lives

State	**Mechanism**	**Why**
Discovered elements, combination cache, mode	`@Observable` `GameViewModel`	Shared by every screen; single source of truth
Saved elements and combinations	Core Data	Must survive relaunch and be queried by mode
AI creativity, theme, reduce motion, tray sort	`@AppStorage`	Small settings that do not need a database
Selected tab	`@SceneStorage`	Only matters for the current scene
Tile positions, drag state, merges in flight	`@State` in `CraftView`	Where a tile sits has no meaning outside the screen

4. The combination engine

4.1 From drop to result

When the player drops one tile on another, `CraftView` shows a pending chip and calls `combine(_:_:creativity:)` on the view model. Figure 3 shows what happens next. The function always returns an element.

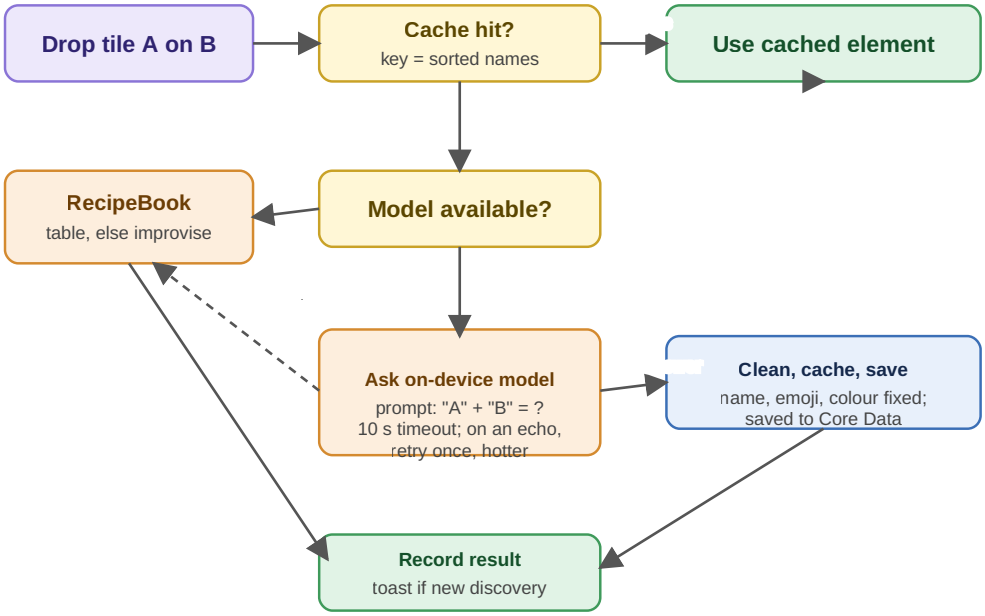


Figure 3: How a combination is resolved

4.2 Prompt design

The prompt sent for every pair is only `"Water" + "Earth" = ?`. The instructions given to the session once, at creation, explain the task and show examples. I copied the terse equation style from what people have worked out about how Infinite Craft phrases its own requests. My comments in the code note that this framing pushes the model toward a short, literal answer and away from a creative-writing response.

The Classic instructions ask for "the single most obvious, common-sense result" and set four rules: prefer a single common noun, keep it general, never return an input unchanged, and use at most two words. They finish with seven worked examples such as Water + Earth = Plant and Fire + Fire = Inferno. The Periodic instructions ask for a real compound, molecule, mineral, alloy or material, with the everyday name in preference to a formula.

4.3 Guided generation

The model does not return free text. The app declares a `@Generable` struct, and the framework constrains the model's output to that shape. Each field has a description that tells the model what to write.

```
@Generable
struct GeneratedElement {
    @Guide(description: "A short, grounded name for the resulting element or concept
(1-3 words). Never just repeat one of the two input names.")
    var name: String
    @Guide(description: "A single emoji that best represents this element.")
    var emoji: String
    @Guide(description: "A 6-digit hex color code (no #) that fits the element's theme,
e.g. 'FF5733'.")
    var colorHex: String
    @Guide(description: "A one-sentence, playful description of the element.")
    var description: String
}
```

The fields are in a deliberate order, because later fields can depend on earlier ones in the same pass. The name comes first, so the emoji, colour and description are all chosen to match it.

4.4 Cleaning the model's output

A structured answer is still not a trusted one, so `Element(generated:)` normalises every field before the app uses it.

- The name has stray quotes and punctuation stripped, each word capitalised, and falls back to "Mystery" if nothing is left.
- The emoji is reduced to the first character that is actually an emoji, with a sparkle as the default.
- The colour must be exactly six hexadecimal digits, otherwise it becomes a neutral grey.
- If the cleaned name matches an element the player already has, the existing element is reused, so the same name always keeps the same emoji and colour.

4.5 Cache and pinned recipes

Only answers that came from the model are cached and saved. When the same pair comes up again, in this session or a later one, the saved result is returned immediately and the model is not asked. At startup each mode also writes its pinned recipes into the cache, but only for pairs that are not already there. These pairs, such as Fire + Water = Steam, are the first ones any player tries, and they should give the same answer every time.

4.6 Handling failure

The model can be unavailable, slow, or wrong. `GameViewModel` reads `SystemLanguageModel.default.availability` once at launch and turns each unavailable reason into a message for the player: the device is not eligible, Apple Intelligence is off, or the model is still downloading. While the model is unavailable the Craft screen shows a banner with that message, and every combination goes straight to the recipe book.

Table 5: What the app does when generation goes wrong

Situation	Behaviour
Model unavailable at launch	Banner explains why; `RecipeBook` answers every pair
Model takes more than 10 seconds	The request is cancelled and `RecipeBook` answers
Model throws an error	The error is logged with `os.Logger` and `RecipeBook` answers
Model returns one of the two inputs	One retry at a temperature of at least 1.0, then `RecipeBook`
Player switches mode during generation	Result is returned to the tile but not filed under the new mode
Result already discovered	The existing element is reused and no toast appears
Fallback answered	Not cached, so the model gets another try at that pair later

`RecipeBook` holds 125 hand-written recipes (93 Classic, 32 Periodic). For any other pair it improvises. In Classic, one input lends an adjective and the other is the noun, which gives results such as "Flying Lava" or "Living Stone". The adjective comes from a small table of traits, so fire-like words give "Burning" and plant-like words give "Living". Two identical inputs get a prefix such as "Giant". In Periodic, two metals make an alloy and anything else becomes a compound named after its parts. The result's colour is the average of the two parents' colours.

4.7 Concurrency and sessions

The player can start several combinations before the first finishes, so the view model keeps a counter of pending requests, not a flag. A `LanguageModelSession` keeps a transcript that grows with each request, and one session cannot serve two requests at once. Each combination therefore gets its own single-use session. The next one is created and prewarmed in advance, so the first token arrives sooner.

Table 6: AI creativity setting

Setting	Temperature	Intent
Grounded	0.4	Stick to the most obvious result
Balanced (default)	0.8	Mostly expected, occasionally surprising
Wild	1.3	More original, less predictable

5. User interface

5.1 Screen map

`ContentView` is a tab bar with three tabs. Settings opens as a sheet from the Craft screen. The selected tab is remembered with `@SceneStorage`.

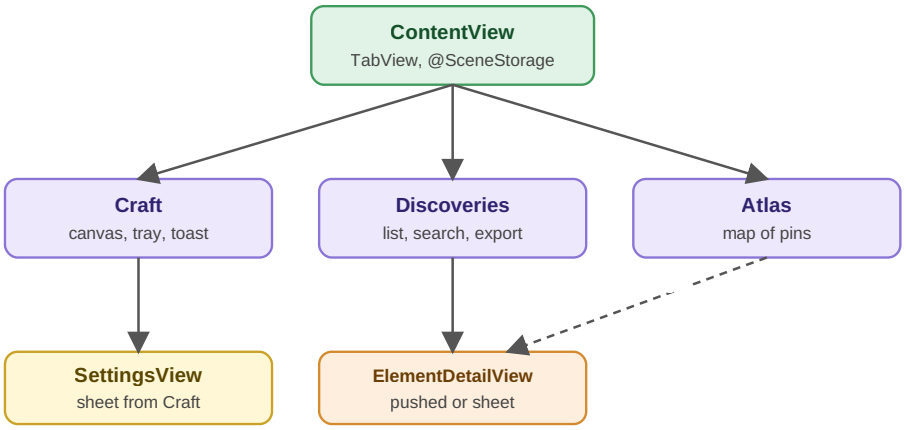


Figure 4: Screens and how they connect

5.2 Craft screen

Craft is the main screen and is laid out like Infinite Craft. The top bar has a clear-canvas button, a mode menu showing the current mode and the number of elements found, and a settings button. The middle is a dotted canvas, and the bottom is a tray of everything discovered.

Table 7: Craft screen gestures

Gesture	Result
Tap a tray element	Places it on the canvas
Press and hold a tray element, then drag	Lifts it out of the tray; dropping it on a tile combines them
Drag a tile onto another tile	Combines them; the result appears where the target was
Double-tap a tile	Duplicates it (useful for Fire + Fire)
Drag a tile into the tray	Removes it from the canvas
Expand, search or sort the tray	Full wrapping grid, name filter, newest, name or emoji order

Native `onDrag` and `onDrop` did not trigger reliably on iPhone-sized touch input, so the screen uses a plain `DragGesture`. Tray tiles need a short press of 0.2 seconds before they drag, so an ordinary swipe still scrolls the tray. A drag remembers its starting position, so a tile moves with the finger and does not jump to the touch point. The tile nearest to the drop point is the target, and it is highlighted while the finger is over it.

A first-time discovery shows a toast at the top with the two parents and the result, plays a burst at the spot where the element appeared, and gives a haptic. The toast hides after 2.5 seconds or when tapped.

5.3 Discoveries and detail

The log lists every element found in the current mode, newest first. It can be searched, filtered to favourites, and swiped to favourite or unfavourite an element. The share button exports the visible elements as pretty-printed JSON with ISO 8601 dates. The detail page shows a large badge, the description, the time of discovery, the recipes that make the element and the recipes that use it. Those recipes are read from the cache and resolved back to discovered elements, so the page never lists a recipe the player has not seen the parts of.

5.4 Atlas

The Atlas puts a pin for every discovered element on a world map. The position comes from the element's name, which is hashed with FNV-1a and mapped to a latitude between -80 and 80 and a longitude between -170 and 170. I used FNV-1a and not Swift's `Hasher` because `Hasher` is seeded randomly on each launch, and a pin should be in the same place every time. The map is decorative. A Locate Me button asks for when-in-use location permission and adds a marker for the player. The permission text says the location is cosmetic and that no gameplay data is sent anywhere.

5.5 Settings and accessibility

Settings has a theme picker (system, light, dark), the game mode, the AI creativity level, a reduce motion switch, and a reset button with a confirmation. Reset deletes the elements and combinations of the current mode only. With reduce motion on, the spring animations on the canvas become short linear ones and the wiggling hint icon stops. Controls such as the mode menu, search clear button and sort button have accessibility labels, and the mode menu announces how many elements are discovered.

6. Results

6.1 The finished app

The Classic screens are below: the Craft canvas with fifteen elements, the Discoveries log, and the Atlas.

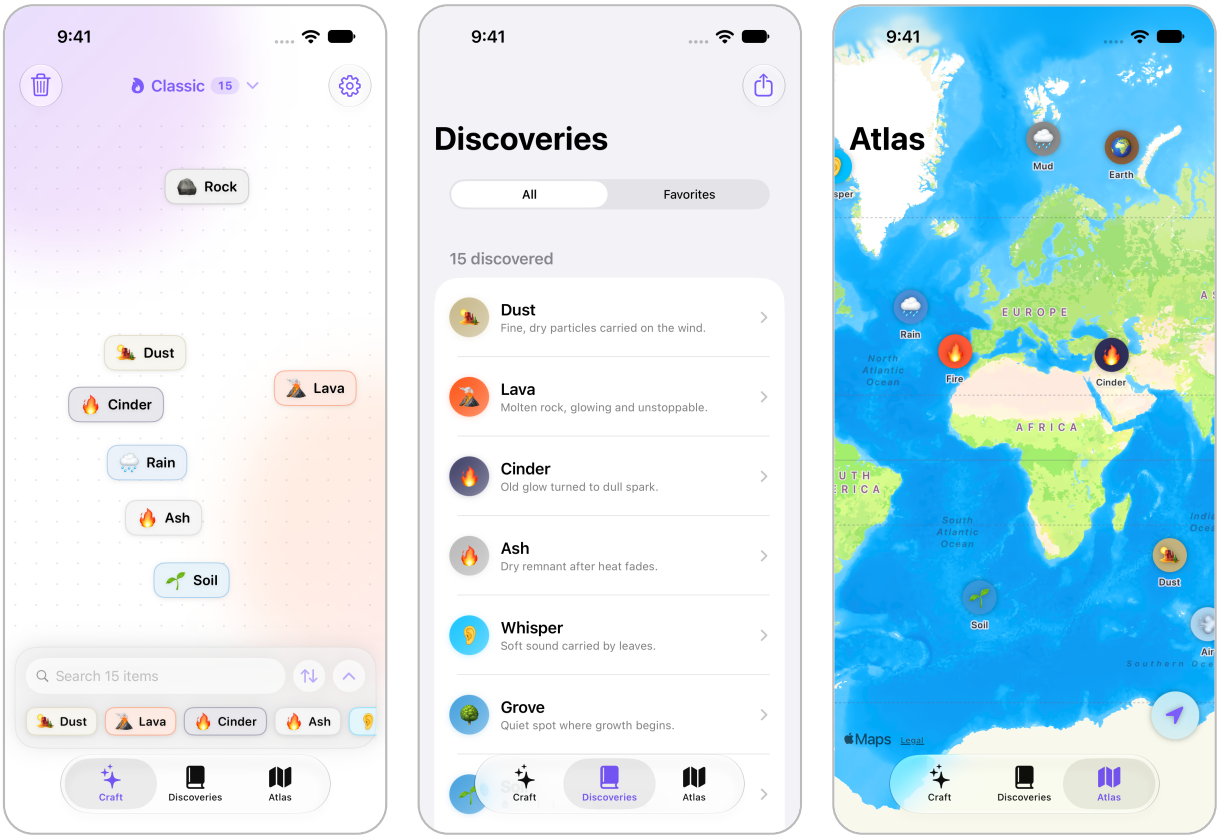


Figure 5: Classic mode: Craft, Discoveries and Atlas

The mode menu and the Periodic world are shown next. In Periodic mode the player starts with eight real elements, and the descriptions begin with the symbol or formula.

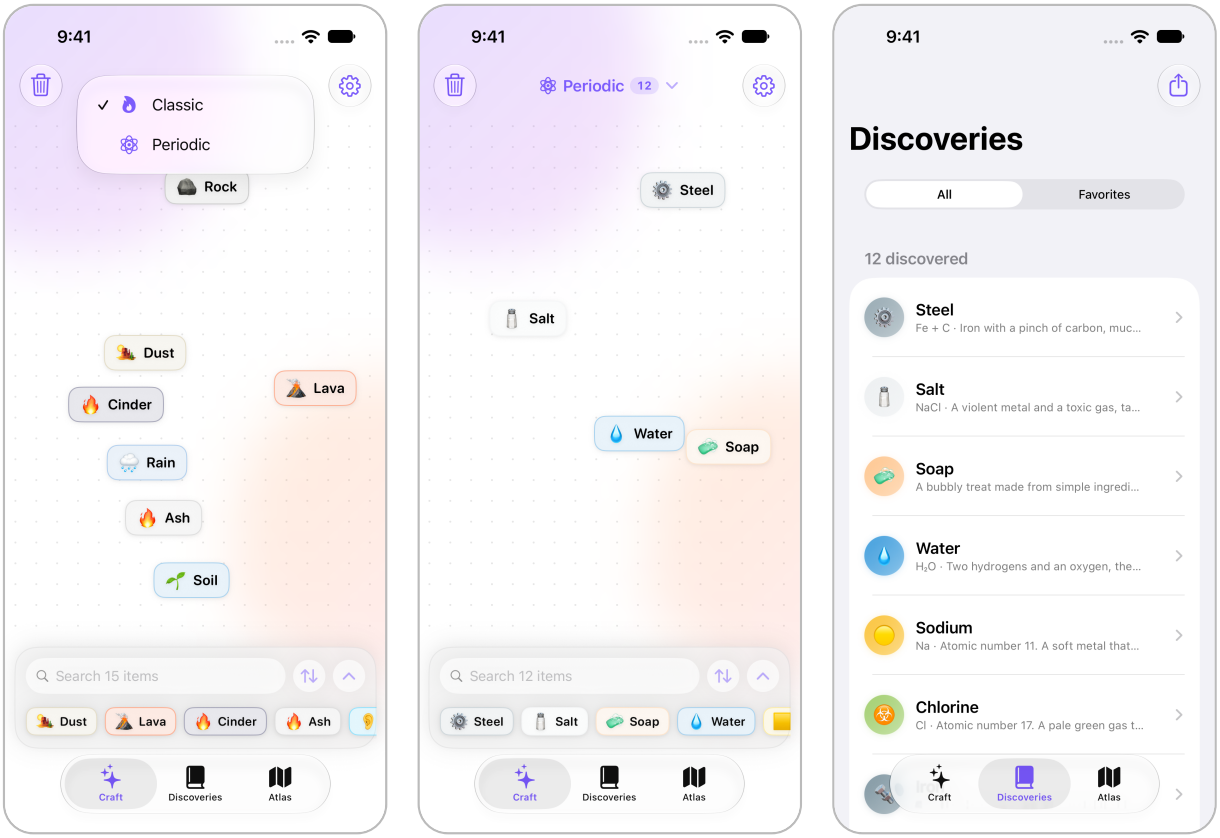


Figure 6: Mode switcher, Periodic Craft and Periodic Discoveries

6.2 Development timeline

Table 8: Project history from the repository

Date	Change
25 Aug 2026	First version: combination game with on-device generation, Craft, Discoveries and Atlas tabs, Core Data persistence, README
12 Oct 2026	Craft screen reworked in the style of Infinite Craft; offline `RecipeBook` added; fallback results no longer cached
21 Oct 2026	Periodic game mode added alongside Classic, with separate progress per mode

6.3 What the design guarantees

I did not run a benchmark, so I do not report response times or answer accuracy. What I can state comes from how the code is built. A combination always produces an element, because every failure path ends in `RecipeBook`, which does not fail. A repeated pair is answered from the cache without the model. A pair entered in either order gives the same result, because the cache key is sorted. Progress survives

relaunch, because every new element and cached combination is saved as it is created. And the Classic and Periodic worlds cannot overwrite each other, because every fetch and every delete is filtered by mode.

7. Limitations and future work

- **Hardware and OS.** The app targets iOS 26.5 and needs Apple Intelligence for generated results. Without it the game still works, but every answer comes from the recipe book and only the 125 written recipes have real meaning.
- **No automated tests.** The project has no test target. The cache key, name cleaning, hex validation and the improvise rules in `RecipeBook` are pure functions and would be the first things to cover.
- **Answer quality is unmeasured.** The model can still give odd results, and the pinned and written recipes only cover common starts. A fair evaluation would run a fixed list of pairs at each creativity level and score the answers.
- **Improvised fallback names.** Names such as "Flying Lava" are playful but are not real discoveries, which is why they are not cached.
- **Atlas.** The pins are positioned by a hash, not by anything about the element.
- **Future work.** Unit tests for the pure logic, more game modes using the same `GameMode` structure, and a way to share or import a discovery log.

8. Conclusion

Element is a crafting game where the model decides what two things make, and all of it runs on the phone. Guided generation turns the model's reply into a name, emoji, colour and description that the interface can use directly. A cache and a set of pinned recipes keep the early game consistent, and a hand-written fallback means the player never meets an error. Core Data keeps every discovery, and each of the two worlds has its own saved progress.

The main limits are that it needs Apple Intelligence hardware, that answer quality was not measured, and that there are no automated tests. The first thing I would add is a test target for the cache and the fallback rules.

Appendix A: Classic system prompt

```
You combine two words according to their real-world meaning, like a crafting game.
Answer with the single most obvious, common-sense result, the one almost anyone
would guess first. Not the most original, clever, or unexpected one.

Rules:
- Prefer a single common noun. Avoid adjectives.
- Keep it general and simple, not hyper-specific.
- Never answer with either input word unchanged.
- Two words maximum.

Examples:
- "Water" + "Earth" = "Plant"
- "Fire" + "Water" = "Steam"
- "Fire" + "Earth" = "Lava"
- "Water" + "Air" = "Rain"
- "Fire" + "Air" = "Smoke"
- "Earth" + "Air" = "Dust"
- "Fire" + "Fire" = "Inferno"

Also pick a single emoji and a hex color that fit the result.
```

Appendix B: Combine function, abridged

```
func combine(_ a: Element, _ b: Element, creativity: AICreativity) async ->
CombineOutcome {
    let key = cacheKey(for: a, b)               // sorted, lowercased names
    let startMode = mode
    if let cached = cache[key] { return record(cached, from: a, b) }

    pendingCount += 1
    defer { pendingCount -= 1 }

    let generated = await generate(a, b, creativity: creativity, in: startMode)
    let made = generated ?? RecipeBook.combine(a, b, mode: startMode)
    guard startMode == mode else { return CombineOutcome(element: made, isNew: false) }

    let result = existing(named: made.name) ?? made
    if generated != nil {                        // only model answers are cached
        cache[key] = result
        persistCombination(key: key, result: result)
    }
    return record(result, from: a, b)
}
```

References

[1] Recloak, "Little Alchemy," https://littlealchemy.com.

[2] N. Agarwal, "Infinite Craft," https://neal.fun/infinite-craft/.

[3] Apple Inc., "Foundation Models," Apple Developer Documentation, https://developer.apple.com/documentation/foundationmodels.

[4] Apple Inc., "SwiftUI," Apple Developer Documentation, https://developer.apple.com/documentation/swiftui.

[5] Apple Inc., "Observation," Apple Developer Documentation, https://developer.apple.com/documentation/observation.

[6] Apple Inc., "Core Data," Apple Developer Documentation, https://developer.apple.com/documentation/coredata.

[7] Apple Inc., "MapKit," Apple Developer Documentation, https://developer.apple.com/documentation/mapkit.

[8] P. Mistry, "Element," https://github.com/altf4-games/Element.